G16 Project 1 Documentation (xv6 Custom System Calls)

1. Project Summary

This project extends xv6-riscv with custom system calls for synchronization, process control, IPC, and process introspection. It includes user-space demo programs for validating each feature inside xv6.
Repository root for the implementation: `xv6-riscv/`.

2. Team Members

- Rachit Kumar Pandey [24JE0677]
- Rahul Joshi [24JE0678]
- Raj Priyadarshi [24JE0679]
- Rajarshi Chakraborty [24JE0680]
- Ramala Karthik [24JE0681]
- Ranish Garg [24JE0682]

3. Features Implemented

3.1 Kernel Mutex Support

- Syscalls: `mutex_create`, `mutex_lock`, `mutex_unlock`
- Demo: `mutexdemo`
- Summary: Kernel-managed mutex table with ownership checks and blocking acquisition behavior.

3.2 Freeze/Thaw Process Control

- Syscalls: `freeze(pid)`, `thaw(pid)`
- Demo: `freezethawdemo`
- Summary: Temporarily pauses and resumes target processes using scheduler-aware process state.

3.3 Message Queue IPC

- Syscalls: `msgget`, `sendmsg`, `recvmsg`
- Demo: `msgdemo`

- Summary: Key-based queue creation/access and typed message passing with non-blocking receive.

3.4 Shared Memory IPC

- Syscalls: `shmget`, `shmat`, `shmdt`
- Demo: `shmdemo`
- Summary: Shared segments with page mapping, refcounting, and auto-cleanup on detach/exit.

3.5 Targeted Child Wait

- Syscall: `waitpid(int pid, int *status)`
- Demo: `waitpiddemo`
- Summary: Parent can wait for a specific child process instead of any child.

3.6 Parent PID Lookup

- Syscall: `getppid(void)`
- Demo: `getppiddemo`
- Summary: Returns caller parent PID with safe fallback behavior.

3.7 Batch Process Creation

- Syscall: `forkn(int n)`
- Demo: `advancedsysdemo`
- Summary: Creates multiple children in one syscall invocation.

3.8 Thread-Style Process Start

- Syscall: `thread_create(void *start, void *arg)`
- Demo: `advancedsysdemo`
- Summary: Starts execution from a function pointer with argument in user mode.

3.9 Non-Blocking Mutex Acquire

- Syscall: `mutex_trylock(int id)`
- Demo: `advancedsysdemo`
- Summary: Immediate lock attempt without sleeping.

3.10 Unified Signal Dispatch

- Syscall: `signal_send(int pid, int sig)`

- Demo: `advancedsysdemo`
- Summary: Unified entry point for TERM/STOP/CONT semantics.

3.11 Message Queue Depth Query

- Syscall: `msgcount(int qid)`
- Demo: `advancedsysdemo`
- Summary: Returns number of pending messages in a queue.

3.12 Mutex Ownership Introspection

- Syscall: `mutex_owner(int id)`
- Demo: `sys2demo`
- Summary: Reports PID owning the mutex, unlocked state, or invalid ID.

3.13 Process State Introspection

- Syscall: `getprocstate(int pid)`
- Demo: `sys2demo`
- Summary: Returns xv6 internal process state for a specific PID.

4. Build and Demo Run

From project root:

```
cd xv6-riscv
make clean && make qemu
```

Then run demos in xv6 shell as needed:

- `mutexdemo`
- `freezethawdemo`
- `msgdemo`
- `shmdemo`
- `waitpiddemo`
- `getppiddemo`
- `advancedsysdemo`
- `sys2demo`

5. Contribution Breakdown

- Rachit Kumar Pandey [24JE0677]

- Feature 1: mutex syscall set (`mutex_create` , `mutex_lock` , `mutex_unlock`)
- Feature 2: process control (`freeze` , `thaw`)
- Ramala Karthik [24JE0681]
- Feature 3: message queue IPC (`msgget` , `sendmsg` , `recvmsg`)
- Ranish Garg [24JE0682]
- Feature 4: shared memory IPC (`shmget` , `shmat` , `shmdt`)
- Raj Priyadarshi [24JE0679]
- Feature 5: `waitpid`
- Feature 6: `getppid`
- Rahul Joshi [24JE0678]
- Feature 7: `mutex_owner`
- Feature 8: `getprocstate`
- Rajarshi Chakraborty [24JE0680]
- Feature 9: `forkn`
- Feature 10: `thread_create`
- Feature 11: `signal_send`

6. Screenshot

- Mutex demo:

```
➜  xv6-riscv git:(main) ✗ make qemu
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 12
8M -smp 3 -nographic -global virtio-mmio.force-legacy=false -drive file=
fs.img,if=none,format=raw,id=x0 -device virtio-blk-device,drive=x0,bus=v
irtio-mmio-bus.0

xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
$ mutexdemo
parent: acquired mutex
child: trying to lock mutex
parent: releasing mutex
child: acquired mutex
child: released mutex
parent: child finished
$
```

- Freeze/thaw demo:

```
➜  xv6-riscv git:(main) ✗ make qemu
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 12
8M -smp 3 -nographic -global virtio-mmio.force-legacy=false -drive file=
fs.img,if=none,format=raw,id=x0 -device virtio-blk-device,drive=x0,bus=v
irtio-mmio-bus.0

xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
$ freezethawdemo
T+39 child: running
T+54 child: running
T+69 parent: child frozen
T+149 parent: thawing child
T+149 child: running
T+164 child: running
T+179 child: running
T+194 child: running
T+209 child: running
T+224 child: running
T+239 child: done
T+239 parent: demo complete
$
```

- Message queue demo:

```
PROBLEMS   OUTPUT   PORTS   TERMINAL   DEBUG CONSOLE                                   make - xv6-riscv

ramala_balaji@LAPTOP-NHPQRAP2:~/os_practice/NCSC210-G16/G16_Project1_xv6CustomizeSystemCalls/xv6-riscv$ make clean && make qemu
balloc: write bitmap block at sector 46
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3 -nographic -global virtio-mmio.force-legacy=false -drive file=fs.img,if=none,format=raw,id=x0 -d
evice virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

hart 1 starting
hart 2 starting
init: starting sh
$ msgdemo
message sent: from child
message received from child: hello parent, how are you
$
```

- Shared memory demo:

```
Ranish@Arch-Linux  ~/Desktop/NCSC210-G16/G16_Project1_xv6CustomizeSystemCalls/xv6-riscv  main ±  make qemu

xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
$ shmdemo
starting shared memory demo
shmget(42, 4096) = 0
parent attached at 0x0000003FFFFFA000
parent wrote: "hello from parent"
child shmget(42, 4096) = 0
child attached at 0x0000003FFFFFA000
child reads: "hello from parent"
child wrote: "hello from child"
child detached
parent reads after child: "hello from child"
parent detached
```

- Waitpid demo:

```
$ waitpiddemo
waitpiddemo: started
parcchheint:ldi1 ch(piid=l5ldd)r2en: c r(peiasdpaw=6nte): spd chiealdwn
ded
c1hild=5 1(pchilid2=6d
paren=t5): : ewxaitiintgi speng with status c1i1f
ically for child1 pid=5
parent: child1 finished, status=11
child2(pid=6): exiting with status 22
parent: child2 finished, status=22
waitpiddemo: complete
$
```

- Getppid demo:

```
$ ls
.              1 1 1024
..             1 1 1024
README         2 2 2425
cat            2 3 38840
echo           2 4 37624
forktest       2 5 19008
grep           2 6 42392
init           2 7 38000
kill           2 8 37552
ln             2 9 37344
ls             2 10 40968
mkdir          2 11 37608
rm             2 12 37592
sh             2 13 61552
stressfs       2 14 38464
usertests      2 15 195472
grind          2 16 54104
wc             2 17 39864
zombie         2 18 36872
logstress      2 19 39624
forphan        2 20 38376
dorphan        2 21 37808
mutexdemo      2 22 38784
freezethawdemo 2 23 38720
msgdemo        2 24 38208
shmdemo        2 25 39832
waitpiddemo    2 26 38656
getppiddemo    2 27 37496
console        3 28 0
$ getppiddemo
getppiddemo: parent pid=4 ppid=2
getppiddemo: child pid=5 ppid=4
getppiddemo: complete
$
```

- Mutex_owner, getprocstate demo:

```
 ~/D/N/G/xv6-riscv  main(!?) ⇝ make qemu                                                    21:56takes 59s  v3.14.3
qemu-system-riscv64 -machine virt -bios none -kernel kernel/kernel -m 128M -smp 3 -nographic -global virtio-mmio.force-legacy=false -drive file=fs.
img,if=none,format=raw,id=x0 -device virtio-blk-device,drive=x0,bus=virtio-mmio-bus.0

xv6 kernel is booting

hart 2 starting
hart 1 starting
init: starting sh
$ sys2demo
sys2demo: begin pid=3
getprocstate(self)=4 (RUNNING)
mutex_owner before lock=0
mutex_owner while locked=3
mutex_owner after unlock=0
signal_send STOP to child=4
child state after STOP=3 (RUNNABLE)
signal_send CONT to child=4
msgcount=1
recvmsg len=7 text=ipc-ok
thread[1] pid=5 ppid=3
joined thread pid=5 status=41
joined child pid=4 status=-1
sys2demo: complete
$
```